

Aula 14 — Complexidade de Algoritmos

Da função de custo ao Big O — com experimentos em Julia + Pluto

A aula constrói a análise de complexidade a partir da pergunta “como o trabalho cresce?”, em vez de apenas “funciona?”, propondo contar passos lógicos — não segundos, que dependem de processador, cache, compilador e outros fatores da máquina — para chegar a uma função de custo $T(n)$ e, só então, ao termo dominante e à notação Big O, deixando claro que o Big O não nega as constantes: apenas escolhe olhar para a região assintótica, de modo que duas funções da mesma classe (como $2n$ e $2000n$) ainda podem ter custos concretos muito diferentes. Sequências de passos calibram o crescimento por meio de constantes aditivas e multiplicativas, condicionais tendem a manter custo constante mas importam para melhor caso, pior caso e caso médio, e laços aninhados multiplicam o crescimento — um laço gera $O(n)$, dois geram $O(n^2)$, três geram $O(n^3)$.

A recursão é tratada por meio do exemplo do Fibonacci: a implementação recursiva ingênua gera uma árvore de chamadas que recalcula repetidamente os mesmos subproblemas, evidenciando que “recursão não é sinônimo de explosão” — recorrências lineares e logarítmicas crescem de forma bem mais contida, e é a estrutura dos subproblemas que determina o crescimento. A programação dinâmica, por meio de memoização, é apresentada como a técnica que elimina essa recomputação, reduzindo o custo do Fibonacci de exponencial para $O(n)$, com um experimento em Julia/Pluto comparando as versões recursiva, memoizada e iterativa. A síntese da aula resume o percurso: o ponto de partida não é o Big O, é a função de custo do algoritmo — laços criam crescimento, sequências o calibram, e a programação dinâmica elimina recomputação.